Unity Editor Extension and Scripting API Reference

38 sources

The provided texts offer a comprehensive overview of Unity Editor Tools, focusing on modern development practices within the Unity environment. They highlight the shift towards UI Toolkit for creating user interfaces, emphasising its visual authoring with UI Builder, event-driven performance, and USS (Unity Style Sheets) for styling, contrasting it with the older IMGUI system. ScriptableObjects are presented as a crucial data management solution for persistence, decoupling, and shareability, while SerializedObject and SerializedProperty facilitate generic, undoable editing of Unity object fields. The sources also discuss different types of custom editor tools, including global and component-specific tools, and practical applications like asset palettes and modular assembly tools, all while stressing the importance of comprehensive documentation.

Discuss ScriptableObjects.

ScriptableObjects are a fundamental data management solution in Unity, particularly important for developing custom Editor tools and managing game data efficiently.

Here's a detailed discussion on ScriptableObjects:

What are ScriptableObjects?

ScriptableObjects are serializable Unity types that are derived from `UnityEngine.Object`. Unlike `MonoBehaviours`, ScriptableObjects are not attached to `GameObjects` as components. Instead, they exist in your project as `.asset` files, independent of any `GameObject` in a scene. Because they inherit from `UnityEngine.Object`, you can drag and drop instances of ScriptableObjects into fields in the Unity Inspector, just like other assets.

Purpose and Core Functionality

The main purpose of a ScriptableObject is to serve as a data store. They are primarily used as containers for shared data, which can be referenced by multiple objects at runtime, thereby reducing memory usage by avoiding unnecessary copies of values. They can also define behavior, although their main value is for data storage.

Key Benefits of ScriptableObjects

The use of ScriptableObjects offers several significant benefits for Unity development:

• **Persistence**: Data stored in ScriptableObjects is saved as `.asset` files within your project, meaning the information is retained across different Unity editor sessions. This ensures that configuration settings or tool data are not lost when the editor is closed.

• **Decoupling**: ScriptableObjects separate tool data from its logic. This allows UI windows or other Editor tools to be closed without any data loss, as the data resides in a separate, persistent asset. This promotes a cleaner architectural design.
• **Shareability**: Multiple tools or systems can easily reference and share the same ScriptableObject data. This is particularly useful for shared configurations or asset lists, as it reduces memory usage by preventing data duplication.

Common Use Cases

ScriptableObjects are widely used in Unity for various purposes, especially in custom Editor tools and for managing shared runtime data.

• **Editor Session Data Storage**: Many authoring tools in Unity, such as `EditorTool` and `EditorWindow`, derive from `ScriptableObject` because they are ideal for saving and storing data during an Editor session.
• **Runtime Asset Data**: They are excellent for saving data as an asset in your project to be used at runtime. For instance, a `SpawnManagerScriptableObject` can store data like `prefabName`, `numberOfPrefabsToCreate`, and `spawnPoints` to determine where to instantiate prefabs at runtime.
• **Custom Editor Tools Examples**:
  ◦ **Unified Asset Palette and Search Tool**: A `SyntyPackData` ScriptableObject can store pack names and references to all its prefabs, along with a `PrefabData` class (serializable within `SyntyPackData`) for detailed prefab information like reference, category, and pack name. This data is persistent and regeneratable by a custom Editor script. A `PrefabPaletteData` ScriptableObject is another example used to store a list of `GameObjects` (prefabs) for a custom asset palette, showcasing data persistence and reusability.
  ◦ **Prefab Painter with Advanced Randomization**: A `PrefabBrush` ScriptableObject stores a list of prefabs and randomization settings such as scale range, rotation, and alignment to surface normals. This allows the tool to rapidly populate scenes with varied environmental details.
  ◦ **Modular Structure Assembly Tool**: A `Building Generator` ScriptableObject can define house layouts, enabling designers to place pre-assembled house variants.
  ◦ **Character Customization and Randomizer**: `CharacterPart` ScriptableObjects can be used to categorize and organize character parts (e.g., body, head, hat, weapons) for generating unique NPCs.

Creating ScriptableObjects

You can create your own custom C# classes that derive from `ScriptableObject`. The quickest way to do this is to use the predefined `ScriptableObject` script template from the **Assets** menu in the Unity Editor.

1. Navigate to **Assets** > **Create** > **Scripting** > **ScriptableObject Script** from the main menu, or right-click in the Project window and select **Create** > **Scripting** > **ScriptableObject Script**.

2. Once you have a custom base class inheriting from `UnityEngine.ScriptableObject`, you can use the `CreateAssetMenu` attribute to allow creating instances of this class directly from the Unity Editor's **Assets** menu. For example:

3. This attribute allows you to create new `.asset` instances of `PrefabPaletteData` by going to **Assets** > **Create** > **My Tools** > **Prefab Palette**.

Saving Changes to ScriptableObjects

While Unity automatically saves changes made to a ScriptableObject asset through the Inspector or Editor authoring tools, direct modifications via script in Edit mode are not automatically saved. To ensure that programmatic changes persist between Editor sessions, you **must** explicitly call `EditorUtility.SetDirty()` on the ScriptableObject. Without this call, the changes will appear in memory but will revert once the Editor is closed and reopened. For example:

```
// Save this as Assets/Editor/GameSettingsEditor.cs
using UnityEditor;
using UnityEngine;

public class GameSettingsEditor : EditorWindow
{
    GameSettings settings;

    [MenuItem("Window/Game Settings Editor")]
    public static void ShowWindow()
    {
        GetWindow<GameSettingsEditor>("Game Settings Editor");
    }

    void OnGUI()
    {
        settings = EditorGUILayout.ObjectField("Settings", settings, typeof(GameSettings), false) as
GameSettings;
        if (settings == null) return;
        EditorGUILayout.LabelField("High Score", settings.highScore.ToString());
        if (GUILayout.Button("Increase High Score"))
        {
            settings.highScore += 10;
            // Call SetDirty to ensure the change is saved
            EditorUtility.SetDirty(settings);
            // Optional: AssetDatabase.SaveAssets(); // To save immediately
        }
    }
}
```

This example shows how `EditorUtility.SetDirty(settings)` is used after modifying `settings.highScore` to ensure the change is saved to the asset on disk.

ScriptableObjects in the Context of Unity Editor Tools

ScriptableObjects are a core architectural principle for building robust, maintainable, and marketable custom Unity Editor tools. They complement other data management techniques like `SerializedObject` and `SerializedProperty`, which are used for generically editing serialized fields on Unity objects and automatically handling undo/redo and Prefab overrides. Together, these components form a powerful pipeline for data management and editing within the Unity Editor.

Would you like me to elaborate on any specific aspect of ScriptableObjects, such as their interaction with `SerializedObjects`, or perhaps demonstrate another use case? I can also quiz you on the material we've covered to test your understanding.

Discuss what these sources say about Unity Editor Tools & UI Toolkit.

Unity Editor Tools and UI Toolkit are central to modern Unity development, enabling developers to streamline workflows, manage complex data, and create intuitive interfaces for both the Editor and runtime applications. Unity 6.2 particularly emphasises the importance of custom tools for enhancing productivity and functionality.

I. Unity Editor Tools: Core Principles

Custom Unity Editor tools are designed to automate repetitive tasks, enhance asset management, and improve overall efficiency in game development, especially with large collections of modular assets where tasks like sorting, placing, and generating variations can be tedious and time-consuming. To ensure these tools are robust, maintainable, and marketable, Unity recommends several core architectural principles.

A. Tool Context and Types

Unity provides the `EditorTool` API for creating custom tools that operate within the Editor. An editor tool's "context" determines what it affects.

• **Tool Context API (`EditorToolContext`):** This API can change what Unity's built-in Transform tools affect. The default context is `GameObject`, which affects a GameObject's Transform values. Other contexts can make tools affect different elements, such as Spline knots and tangents in a Spline tool context. If multiple tool contexts exist in a project, users can select between them via the Tools overlay.

• **Global Tools vs. Component Tools:** Tools created with the `EditorTool` API can be categorised into two main types:

◦ **Global Tools** affect any `GameObject` and are always available regardless of the selected `GameObject` type. Examples include the built-in Transform tools like Move, Rotate, Scale, and Rect. They are displayed in the Tools overlay after the built-in Transform tools.

◦ **Component Tools** affect a specific `Component` and are only available when a `GameObject` with that associated `Component` is selected. For instance, a custom manipulator tool for lights would only appear when a `GameObject` with a Light component is selected. They appear as the last buttons in the Tools overlay, grouped by their component.

B. Data Management

Effective data management is crucial for robust editor tools.

• **ScriptableObjects:** These are essential for storing tool data, such as asset lists or configuration settings, ensuring persistence and separation of concerns. They are serializable Unity types that exist as assets in the project, independent of GameObjects. Key benefits include persistence (data saved as `.asset` files across sessions), decoupling (separating tool data from logic), and shareability (multiple tools can reference the same data, reducing memory usage). They are commonly used as containers for shared runtime data and for saving/storing data during an Editor session. When modifying ScriptableObjects via script in Edit mode, `EditorUtility.SetDirty()` must be called to ensure changes are saved to disk.
• **SerializedObject and SerializedProperty:** These classes are fundamental for generically editing serialized fields on Unity objects and are the recommended approach for creating custom Editors. They automatically handle "dirtying" individual serialized fields, ensuring they are processed by the Undo system and correctly styled for Prefab overrides in the Inspector. `SerializedObject` allows for editing multiple target objects simultaneously. Changes made via `SerializedProperty` within a `SerializedObject` stream must be flushed using `SerializedObject.ApplyModifiedProperties()` to persist.

II. UI Toolkit: Unity's Modern UI Framework

UI Toolkit is Unity's modern UI framework, designed to replace both the older Immediate Mode GUI (IMGUI) for editor interfaces and eventually Unity UI (uGUI) for runtime interfaces. It is heavily influenced by web technologies like CSS and HTML, but is tailored for Unity's specific needs.

A. Key Advantages over IMGUI

UI Toolkit offers significant advantages over IMGUI:

• **Authoring:** UI Toolkit supports visual authoring using the **UI Builder**. This separates visuals (UXML/USS) from logic (C#), similar to HTML/CSS. In contrast, IMGUI requires UI to be entirely coded in C#, leading to less readable and maintainable UIs for complex tools.
• **Performance:** UI Toolkit is event-driven, only redrawing elements when changes occur, making it more efficient. IMGUI, on the other hand, redraws the entire UI every frame, which is resource-intensive.
• **Styling:** UI Toolkit uses **Unity Style Sheets (USS)** for flexible and powerful styling, promoting consistent design and supporting editor variables for themes (e.g., light/dark mode) and animations. IMGUI relies on line-by-line code styling, hindering consistent design.
• **Best Use Cases:** UI Toolkit is ideal for polished, complex, and professional editor tools for large projects. IMGUI is better suited for simple debug windows or quick inspectors.

B. UI Toolkit Components and Features

• **UI Builder:** This is a visual authoring tool within the Unity Editor used for creating and editing UI Documents (`.uxml`) and StyleSheets (`.uss`). It supports drag-and-drop UI creation and real-time preview. When authoring editor extensions in UI Builder, you must enable "editor extension authoring" to access additional editor controls and themes.

• **UXML (Unity Extensible Markup Language):** An XML-based asset that defines the structure and layout of UI elements, similar to HTML. You can create UXML documents visually with the UI Builder or by text editing. UXML documents can be used as templates within other UXML documents, similar to Prefabs.
• **USS (Unity Style Sheets):** CSS-like text files used for flexible and powerful styling of UI elements. USS supports editor variables for consistent theming (e.g., light/dark mode) and animations. While you can't create new USS variables in UI Builder, you can assign existing ones to style properties.
• **VisualElement:** The base class for all UI Toolkit elements, serving as a container and parent for other UI components. The UI Toolkit visual tree is composed of these lightweight nodes and is virtual, not using GameObjects for each element, which removes overhead.
• **Data Binding:** This system links UI controls to object properties or serialized properties, allowing for automatic display and update of values in both directions. Unity 6.1 introduced enhanced data binding, especially for runtime UI. Unity 2023 (or Unity 6.1 for runtime) features a new binding system allowing `DataSource` and `BindingPath` attributes to work without extensive manual C# code for runtime UI.
• **Events:** UI Toolkit provides an event system for handling user interactions and notifications, with terminology and naming similar to HTML events. Callbacks can be registered for various events, such as button clicks or value changes.
• **Custom Property Drawers & Editors:** UI Toolkit allows direct implementation of custom property drawers and full custom editors, providing granular control over how data is presented and edited in the Inspector.
• **Live Reload:** A feature in UI Toolkit that provides immediate UI updates in the editor during development, even without saving UXML/USS files. This feature is incredibly convenient but can make it easy to forget to save.
• **Editor Foundations Design System:** A resource providing guidance, best practices, and other resources for building better Editor UIs, promoting consistency with Unity's native interface.
• **Reusable Components:** Developers can create reusable UI components that encapsulate behaviour and interactions by extending `VisualElement` and defining UI in UXML, saving significant development time. Custom controls often involve inheriting from `VisualElement` (or `BindableElement` for data binding), defining `UXMLFactory` for tag exposure in UXML, and `UXMLTraits` for attributes.
• **Pre-creating Hierarchies and Pooling:** To improve performance, especially with many elements, it's recommended to pre-create elements and hide them with `display:none` when not needed, and to pool recurring elements rather than instantiating them with `new()` every time. Using `ListView` is recommended to keep the number of visible elements low, as it pools and recycles elements as the user scrolls.
C. UI Toolkit vs. uGUI (Unity UI)

While UI Toolkit is designed to replace uGUI for runtime UI eventually, they have distinct approaches.

• **UI Hierarchy:** Both use a hierarchy tree structure. In uGUI, elements are `GameObjects` visible in the Hierarchy view, while in UI Toolkit, `VisualElements` are organised in a virtual visual tree not directly visible in the Hierarchy view. The UI Debugger helps view and debug the UI Toolkit hierarchy.

• **Canvas vs. UIDocument:** The `Canvas` component in uGUI is similar to `UIDocument` in UI Toolkit, both being `MonoBehaviours` attached to GameObjects. `Canvas` determines sort order, rendering, and scaling. `UIDocument` contains a reference to a `PanelSettings` object for rendering settings and the UXML file defining the UI layout. For Editor UI, no `UIDocument` component is needed; `EditorWindow` classes implement `CreateGUI()` instead.

• **UI Element Access:** In uGUI, scripts access UI elements by assigning references in the Editor or finding components in the hierarchy. In UI Toolkit, there are no GameObjects or components for UI elements, so references are resolved at runtime using query functions on the root `VisualElement` (e.g., `rootVisualElement.Query("childName")` or `rootVisualElement.Query<Button>()`), similar to web development's `document.querySelector`.

• **UI Creation and Reusability:** uGUI saves UI inside Prefabs, often with logic scripts. UI Toolkit separates logic and layout, using UXML files for layout and custom controls for reusability.

• **UI Layout:** uGUI's layout is manual, with elements free-floating and affected by direct parents, using pivots and anchors. UI Toolkit's layout is automatic and influenced by web design, where an element's size and position affect siblings.

• **Events:** Both systems use events for user interactions. In uGUI, events can be exposed in the Editor and set up on GameObjects. In UI Toolkit, callbacks must be set up and handled via scripting, as logic and layout are separate. The event dispatching system in UI Toolkit also differs, as events can be sent to target controls and all parent controls.

• **Mixed Usage:** It is possible to use uGUI and UI Toolkit in the same project, but advanced interactions between mixed UIs, such as keyboard navigation or embedding hierarchies, are not directly supported and would require custom C# scripting to bridge. Styling and event conventions are different, making it difficult to achieve a unified feel.

III. Practical Applications: High-Impact Custom Editor Tools

The architectural principles of Unity Editor tools and UI Toolkit are applied to streamline workflows, especially with modular asset packs like Synty Studios.

• **Unified Asset Palette and Search Tool:** This tool serves as a central hub for discovering, organizing, and quickly accessing all assets within a project, eliminating manual folder navigation. It utilises `SyntyPackData` and `PrefabData` ScriptableObjects for persistent, regeneratable data. The UI Toolkit implementation includes an `EditorWindow`,

`ToolbarSearchField` for filtering, `TwoPaneSplitView` for layout, and `ListView` for displaying prefabs with asynchronous thumbnails.

• **Prefab Painter with Advanced Randomization:** This tool rapidly populates scenes with environmental details (e.g., rocks, bushes) from various packs, ensuring natural variation. It uses a `PrefabBrush` ScriptableObject to store prefabs and randomization settings. The tool handles brush preview and mouse input in `OnSceneGUI` and can instantiate random prefabs with configured settings. It supports "Mixed Brushes" and a "Brush Generator" feature within the Asset Palette.

• **Modular Structure Assembly Tool:** This tool streamlines the assembly of modular structures by automatically snapping pieces together based on defined connection points. Connection points are defined on modular prefabs, and an editor script tracks selected prefabs for snapping. A `Building Generator` ScriptableObject can define house layouts for pre-assembled variants. The UI Toolkit implementation features an `EditorWindow` with a drag-and-drop area, snapping controls, connection point visualization, and a "Build Mode" button activating `OnSceneGUI` interactions.

• **Character Customization and Randomizer:** This tool quickly generates unique Non-Player Characters (NPCs) by mixing and matching modular character parts. It can use `CharacterPart` ScriptableObjects to categorize and organise parts. A "Randomize" button assembles new characters, with an option to generate new Prefab Variants for persistent storage. The UI Toolkit interface includes an `EditorWindow` to display parts, allow mix-and-match on a preview character, and feature a "Randomize" button.

IV. Additional Tools and Concepts

• **EditorWindow:** A class for creating custom editor windows that are available at all times, independent of selected GameObjects, unlike Inspector windows.

• **NaughtyAttributes:** An open-source Unity Inspector extension that expands the range of attributes (e.g., `ReorderableList`, `Button`, `InfoBox`, `ProgressBar`, `Foldout`) without requiring custom editors or property drawers. It can also apply to non-serialized fields or functions.

• **Game Design Document (GDD):** While not an editor tool itself, a comprehensive GDD is essential for managing game development, outlining all aspects of a game. Custom editor tools are designed to enhance the development process outlined in such documents.

Would you like to explore how to build a custom UI Toolkit `EditorWindow`, discuss the requirements for Asset Store submission, or perhaps test your understanding of these concepts with a quiz?

Discuss what these sources say about Additional Tools and Concepts, in the larger context of Unity Editor Tools & UI Toolkit.

Unity Editor Tools and UI Toolkit are powerful components of the Unity development ecosystem, providing extensive capabilities for creating custom workflows and user interfaces. Beyond the

core principles of UI Toolkit, data management, and tool types, several additional tools and concepts further enhance development.

I. Additional Tools and Concepts

This section covers supplementary tools and important concepts that support robust Unity development.

This document provides a comprehensive overview of Unity Editor Tools and the UI Toolkit, focusing on modern development practices within the Unity environment. It highlights the shift towards UI Toolkit for creating user interfaces, emphasizing its visual authoring with UI Builder, event-driven performance, and Unity Style Sheets (USS) for styling, contrasting it with the older IMGUI system.

The document also discusses:

- **ScriptableObjects**: A crucial data management solution for persistence, decoupling, and shareability, used for storing tool data and runtime asset data. Changes to ScriptableObjects via script require `EditorUtility.SetDirty()` to be called.
- **SerializedObject and SerializedProperty**: Essential for generic, undoable editing of Unity object fields.
- **Types of Custom Editor Tools**: Categorized into Global Tools (affect any GameObject and always available) and Component Tools (affect a specific Component and only available when that Component is selected).
- **Practical Applications**: Examples of high-impact custom editor tools for modular assets like those from Synty Studios, including a Unified Asset Palette and Search Tool, Prefab Painter, Modular Structure Assembly Tool, and Character Customization and Randomizer. These tools demonstrate the application of UI Toolkit for interfaces and ScriptableObjects for data.
- **Additional Tools and Concepts**: Covers tools like NaughtyAttributes (Inspector extension), EditorWindow (custom editor windows), and the use of UI Toolkit for runtime UI. It also touches on Game Design Documents (GDD) and best practices for UI Toolkit element management (pre-creating hierarchies, pooling elements, keeping visible elements low).
- **Commercial Release**: Outlines the process and requirements for publishing tools on the Unity Asset Store, including publisher registration, asset submission, creating a professional store page, comprehensive documentation, and legal considerations.

A. NaughtyAttributes

NaughtyAttributes is an open-source Unity Inspector extension designed to significantly expand the range of attributes available to developers. It enables the creation of powerful inspectors without the need for custom editors or property drawers. This extension offers various attributes, including:

• `ReorderableList`.

• `Button`.

• `InfoBox`.

• `ProgressBar`.

• `Foldout`. It also provides attributes that can be applied to non-serialized fields or functions. While most `NaughtyAttributes` function with Unity's `CustomPropertyDrawer`, certain "Meta Attributes" and specific "Drawer Attributes" (like `ReorderableList`, `Button`, `ShowNonSerializedField`, and `ShowNativeProperty`) may require inheriting from `NaughtyInspector` and using `NaughtyEditorGUI.PropertyField_Layout` for full compatibility in custom editors.

B. Reusable Components with UI Toolkit

UI Toolkit facilitates the creation of reusable components that encapsulate behaviours and interactions, significantly reducing development time. This is achieved by extending the `VisualElement` class (or `BindableElement` for data binding) and defining the UI structure in UXML. To integrate custom reusable components effectively:

• **`UXMLFactory`:** Define this to expose the component as a tag in UXML documents and make it available in the UI Builder.

• **`UXMLTraits`:** Use this class to define attributes for the component that can be set in UXML, similar to HTML attributes.

• **Public Properties:** Expose public properties in C# that correspond to the `UXMLTraits` (using cam case) for proper functionality within the UI Builder Inspector. Developers can create these components either visually in the UI Builder or entirely through C# code. When a reusable component is dragged into the UI Builder, it typically uses a default constructor, so an `init` or similar function can be used to perform necessary setup. This approach contrasts with older UI frameworks like uGUI, where UI elements were GameObjects with attached MonoBehaviour scripts, offering a different but ultimately more flexible workflow for managing UI. The UI Toolkit workflow, influenced by web development, encourages thinking of UI as a "visualization for your data" and excels in scenarios with procedurally generated UI or when elements are accessed by name.

C. EditorWindow

The `EditorWindow` class is crucial for creating custom editor windows that are persistently available within the Unity Editor, regardless of which GameObjects are currently selected. This differs significantly from Inspector windows, which are context-dependent and only display information related to the currently selected objects. Custom UI for an `EditorWindow` is typically instantiated within its `CreateGUI()` function, which is particularly beneficial for integrating with the UI Toolkit's "live reload" feature. An `EditorWindow` can also utilize an `InspectorElement` to bind and display properties of a target object within its interface.

D. Runtime UI with UI Toolkit

While primarily known for Editor UI, UI Toolkit is also designed to eventually replace Unity UI (uGUI) for runtime interfaces, building upon web technologies like CSS and HTML. Unity 2023 (or Unity 6.1 for runtime) introduced a new binding system that allows `DataSource` and

`BindingPath` attributes to work seamlessly at runtime without requiring extensive manual C# code. Key aspects of runtime UI Toolkit:

• **`UIDocument` Component:** Runtime UI is driven by this `MonoBehaviour` component, which sits at the root of a visual tree, references a `PanelSettings` object for rendering, and loads the UXML file defining the UI layout.

• **Data Binding:** The `DataSource` attribute on a `VisualElement` specifies the data source (e.g., `this` for the current `MonoBehaviour`), and `BindingPath` attributes defined in UI Builder can be reused at runtime. The `SetBinding` API provides fine-grained control over binding, including bi-directional options and specifying which property (not just `value`) to bind.

• **Styling:** While Editor theme variables do not exist in the runtime theme by default, they can be redefined in runtime stylesheets to maintain a consistent look.

• **Migration from uGUI:** UI Toolkit utilizes a virtual visual tree for UI elements, which, unlike uGUI's GameObject-based hierarchy, reduces overhead but requires using the UI Debugger for inspection. Accessing UI elements in UI Toolkit is done via query functions on the root `VisualElement` (e.g., `rootVisualElement.Query("childName")`), as direct references or `GetComponentInChildren<T>()` are not applicable. Event handling also differs, with callbacks and an event dispatch system that can send events to parent controls.

• **Mixing UI Frameworks:** It is possible to use uGUI and UI Toolkit in the same project, for example, using uGUI for 3D world space UI and UI Toolkit for modal windows. However, advanced interactions such as keyboard navigation between elements from different frameworks or embedding one framework's UI within the other's hierarchy are not supported. Differing styling conventions can also make achieving a unified look challenging.

E. Game Design Document (GDD)

While not an Editor tool itself, a Game Design Document (GDD) is a comprehensive and indispensable document for managing all aspects of game development. It typically covers a wide array of topics, including:

• **Game Description:** Design goals, influences, target market.

• **Functional Specifications:** Core gameplay, game flow, characters/units, gameplay elements, physics, AI, multiplayer.

• **User Interface:** Flow charts, GUI objects.

• **Art and Video:** Overall goals, 2D/3D art, GUI, marketing art, terrain, special effects, cinematics, assets pipeline.

• **Sound and Music:** Overall goals, sound effects.

• **Story:** Player, secondary, and enemy characters, story theme and outline.

• **Level Requirements:** Level diagrams, asset revelation schedule, level design seeds.

• **Technical Specifications:** Game engine, platform, external code, code objects, control loop, game object data, data flow, AI.

• **Production Schedule:** Scope, scheduling, dependencies, cost estimate. A well-structured GDD is essential for guiding development, and custom editor tools are designed to streamline and enhance the various processes outlined within such a document.

F. Other Third-Party and Community Assets

The Unity Asset Store provides a wealth of resources that can significantly accelerate game development, with many developers leveraging community contributions. Some highly recommended assets include:

• **DOTween (Pro):** A powerful tweening library used for programmatically animating "literally anything". It is often one of the first assets added to projects.

• **Odin Inspector:** Praised for simplifying prototyping and building editor tools. However, its studio license agreement has been noted as potentially "predatory".

• **Rewired:** Recommended as an easier solution for adding controller input support to games, often preferred over Unity's native input system due to its perceived complexity.

• **FMOD for Unity:** A powerful audio solution with a free tier for indie developers, offering features like adaptive audio, live editing, and material-dependent sound effects.

• **Animancer:** Provides script-driven animation control, allowing developers to bypass Mecanim trees and integrating well with ScriptableObjects. A free editor-mode trial is available.

• **Inventory Engine by More Mountains:** Offers a solid and easily extensible base inventory system.

• **Feel by More Mountains:** Saves significant development time by facilitating the chaining of effects and is highly extensible.

• **UModeler by Tripolygon:** Considered superior to ProBuilder by some for rapid blocking out and greyboxing of levels to scale.

• **Dialogue System by Pixel Crushers:** Widely regarded as the "best" dialogue solution.

• **Easy Credits 2 by Onerat Pty:** A convenient tool for generating nice-looking credits scenes and importing credit information from spreadsheets.

• **Visual State Machine by Illumisoft:** A simple, lightweight, and easily extensible visual state machine solution.

• **Look Animator by Fimpossible Creations:** Effective for achieving smooth character gaze at targets without odd deformations.

• **Sensor Toolkit by Micosmo:** A highly functional sensor kit that "just works".

• **UMotionPro:** Invaluable for fine-tuning existing animations and creating simple ones for specific scenes.

• **GAIA, Crest, Sky Master ULTIMATE:** Environment creation tools that assist with terrain stamping, cartoon water, and versatile sky generation respectively.

• **Playmaker:** A popular visual scripting asset for gameplay mechanics.

G. UI Toolkit Element Management Best Practices

To optimize performance and maintain stability when managing elements within the UI Toolkit's visual tree, several best practices are recommended:

• **Pre-create Hierarchies:** Creating and loading UI elements in a hierarchy can be slow. To mitigate this, elements can be pre-created and hidden using `display:none`, then displayed only when needed. However, this may consume more memory if many elements are created simultaneously.
• **Pool Recurring Elements:** Instead of repeatedly instantiating new elements (`new()`) and releasing them, elements pooling involves retaining elements for later reuse. It is crucial to have full control over pooled elements and reset them properly before returning them to the pool to prevent instability, especially concerning event callbacks or non-serialized states.
• **Keep Visible Elements Low:** For displaying large lists of data efficiently, using `ListView` is recommended, as it pools and recycles elements as the user scrolls, ensuring only visible elements are rendered. Developers can also implement custom pooling mechanisms, managing the visible area using `GeometryChangedEvent` and `VisualElement.layout`.
• **Painter2D:** For drawing custom shapes (e.g., arcs, fills) within UI Toolkit's `generateVisualContent` callback (available in Unity 2022.2+), `Painter2D` offers a higher-level API compared to manual mesh generation, simplifying the process and producing smoother edges.
Would you like to explore how to implement any of these specific tools or concepts, or perhaps learn more about the Unity Asset Store publishing process?
Discuss what these sources say about Types of Custom Editor Tools, in the larger context of Unity Editor Tools & UI Toolkit.
Hello! I'm here to be your personal guide to the knowledge base you've provided, which covers various aspects of Unity Editor Tools and UI Toolkit, including core architectural principles, specific tool types, and even considerations for commercial release. Think of me as your helpful expert, ready to assist you in understanding all the details quickly and effectively. I can help you with summaries, specific facts, clarifications, and even quiz you to test your understanding.
Let's delve into the types of Custom Editor Tools within the Unity Editor Tools & UI Toolkit ecosystem. Custom Editor tools are developed to streamline workflows, automate repetitive tasks, enhance asset management, and improve overall efficiency in game development within the Unity environment. They are particularly valuable when managing large collections of modular assets, which often involve tedious tasks like sorting, placing, and generating variations of numerous small prefabs.
Unity categorises custom editor tools primarily by their context and scope, leveraging the EditorTool API.
I. Categories of Custom Editor Tools
1. **EditorTool API** Unity provides the EditorTool API for creating custom tools that operate within the Editor. An Editor tool's *context* determines what that tool affects.
    ◦ **Tool Context (EditorToolContext API)**: This API is used to change what Unity's built-in Transform tools affect.

▪ **Default Context**: The default context is `GameObject`, meaning a tool will affect a GameObject's Transform values. This is known as the `GameObject Tool Context`.

▪ **Other Contexts**: Tools can be designed to affect different elements. For instance, a Spline tool context enables the Move, Rotate, and Scale tools to affect Spline knots and tangents instead of entire GameObjects.

▪ **Selection**: If a project contains multiple tool contexts, users can select between them via the Tools overlay. If the tool context button isn't selected, the default GameObject tool context remains active.

2. **Global Tools vs. Component Tools** Tools created with the EditorTool API can be classified as either global or component tools, depending on their availability and the objects they affect.

◦ **Global Tools**:

▪ These tools affect any GameObject in the scene.

▪ They are always available regardless of the type of GameObject currently selected.

▪ Examples include Unity's built-in Transform tools: Move, Rotate, Scale, and Rect.

▪ Global tools are typically displayed in the Tools overlay after the built-in Transform tools.

◦ **Component Tools**:

▪ These tools affect a specific Component.

▪ They are only available when a GameObject with the associated Component is selected. For instance, a custom manipulator for lights would only appear when a GameObject with a Light component is selected.

▪ Component tools are displayed as the last buttons in the Tools overlay, grouped by their component. Their availability depends on what is actively selected in the Scene view or Hierarchy window.

II. Practical Examples of Custom Editor Tools (High-Impact Tools)

The sources propose several high-impact custom editor tools, particularly for streamlining workflows with modular asset packs like Synty Studios, demonstrating how these architectural principles are applied in practice. These tools often leverage UI Toolkit for their user interfaces and ScriptableObjects for data management.

1. **Unified Asset Palette and Search Tool**

◦ **Purpose**: This tool serves as a central hub for discovering, organizing, and quickly accessing all assets within a project, thereby eliminating the need for manual folder navigation.

◦ **Data Architecture**: It uses a `SyntyPackData` ScriptableObject to store pack names and prefab references, and a `PrefabData` class (a serializable class within `SyntyPackData`) for prefab details (reference, category, pack name). This data is persistent and can be regenerated by a custom Editor script.

◦ **UI Toolkit Implementation**: The main UI is an `EditorWindow`. It features a `ToolbarSearchField` for filtering, a `TwoPaneSplitView` to display packs and a prefab grid, and a `ListView` for the prefab grid, with custom `makeItem` and `bindItem` functions. `AssetPreview.GetAssetPreview()` is used to generate asynchronous thumbnails.

2. **Prefab Painter with Advanced Randomization**

◦ **Purpose**: This tool is designed to rapidly populate scenes with environmental details (e.g., rocks, bushes) from various asset packs, ensuring natural variation.

◦ **Data and Implementation**: It uses a `PrefabBrush` ScriptableObject to store a list of prefabs and randomization settings such as scale range, rotation, and alignment to surface normal. `OnSceneGUI` is implemented in the main Editor script to handle brush preview and mouse input, instantiating random prefabs with configured settings.

◦ **Synty Specifics**: It supports "Mixed Brushes" (combining props from different packs) and a "Brush Generator" feature within the Asset Palette tool to create `PrefabBrush` ScriptableObjects by dragging selected assets.

3. **Modular Structure Assembly Tool**

◦ **Purpose**: This tool streamlines the assembly of modular structures (like walls, floors, and roofs) by automatically snapping pieces together based on defined connection points. This is particularly useful for packs like Dungeon, Fantasy Kingdom, and Office.

◦ **Data and Implementation**: Connection points are defined on modular prefabs using empty GameObjects with specific tags or custom MonoBehaviour components. An Editor script tracks selected prefabs and snaps new pieces to the nearest available connection point. A `BuildingGenerator` ScriptableObject can define house layouts for pre-assembled variants.

◦ **UI Toolkit Implementation**: An `EditorWindow` features a drag-and-drop area for modular pieces, controls for snapping settings, and visualization of connection points. A "Build Mode" button activates special scene view interactions handled by `OnSceneGUI`.

4. **Character Customization and Randomizer**

◦ **Purpose**: This tool quickly generates a large number of unique Non-Player Characters (NPCs) by mixing and matching modular character parts from character packs.

◦ **Data Architecture and Implementation**: It involves a system for categorizing character parts (e.g., body, head, hat, weapons), possibly using `CharacterPart` ScriptableObjects to organize and reference them. A "Randomize" button assembles new characters by randomly selecting parts from each category, with an option to generate new Prefab Variants for persistent storage.

◦ **UI Toolkit Implementation**: An `EditorWindow` displays available character parts, allows users to mix and match on a preview character, and features a "Randomize" button to trigger assembly.

III. Additional Tools and Concepts for Custom Editor Tool Creation

Beyond the direct classification, several other Unity features and concepts are crucial for developing effective custom editor tools, often integrated with UI Toolkit:

• **EditorWindow**: This class is used to create custom editor windows that are available at all times in the Unity Editor, regardless of the currently selected GameObjects. This differs from Inspector windows, which are context-dependent.

• **Custom Property Drawers & Editors**: UI Toolkit allows for direct implementation of custom property drawers and full custom editors. This provides granular control over how data is presented and edited in the Inspector. `SerializedObject` and `SerializedProperty` are the recommended approach for creating custom Editors, as they automatically handle dirtying, Undo system integration, and correct styling for Prefab overrides.
• **NaughtyAttributes**: This is an open-source Unity Inspector extension that expands the range of attributes (e.g., `ReorderableList`, `Button`, `InfoBox`, `ProgressBar`, `Foldout`) available without requiring custom editors or property drawers for many common customizations.
• **Reusable Components**: UI Toolkit facilitates the creation of reusable UI components by extending `VisualElement` and defining UI in UXML. These components encapsulate specific UI hierarchies, behaviours, and interactions, which saves significant development time.
• **Data Binding**: UI Toolkit includes a data binding system that links UI controls to object properties or serialized properties, enabling automatic display and updates of values in both directions. Unity 6.1 introduced enhanced data binding, especially for runtime UI. Unity 2023 further refined this with a new binding system for runtime UI, allowing `DataSource` and `BindingPath` attributes to work without extensive manual C# code.
• **ScriptableObjects**: These are serializable Unity types that exist as assets in the project, independent of GameObjects, and are crucial for storing tool data like asset lists or configuration settings. They offer persistence (data saved as .asset files), decoupling (separating data from logic), and shareability (multiple tools referencing the same data), making data management robust. When modifying a ScriptableObject via script in Edit mode, `EditorUtility.SetDirty()` must be called to ensure changes persist.
These types and concepts highlight Unity's modern approach to editor extension, emphasizing visual authoring and efficient data management to create powerful and professional custom tools.
Would you like to explore any of these specific types of custom editor tools or their underlying concepts in more detail, such as how to implement a `Unified Asset Palette` using `UI Toolkit`? I can also quiz you on the material we've just covered if you like!
Discuss what these sources say about Global Tools vs. Component Tools, in the larger context of Types of Custom Editor Tools.
Within the larger context of Unity Editor Tools and UI Toolkit, Unity provides a clear categorisation for custom editor tools, primarily based on their context and scope. This helps developers create intuitive and efficient workflows. The distinction between Global Tools and Component Tools is a key aspect of this categorisation, leveraging the EditorTool API.
Understanding Custom Editor Tools and Their Context
Unity's **EditorTool API** is the foundation for creating custom tools that operate within the Unity Editor. These tools are designed to streamline workflows, automate repetitive tasks, enhance asset management, and improve overall efficiency in game development.

The *context* of an Editor tool is crucial, as it determines what elements the tool affects. The **EditorToolContext API** allows developers to change what Unity's built-in Transform tools—such as Move, Rotate, and Scale—affect.

• **Default Context**: The standard or default context is `GameObject`, meaning a tool will primarily affect a GameObject's Transform values. This is also known as the `GameObject Tool Context`.

• **Other Contexts**: Tools can be designed to operate within different contexts. For example, a Spline tool context enables the Move, Rotate, and Scale tools to manipulate Spline knots and tangents instead of entire GameObjects.

• **Selection**: If a project incorporates multiple tool contexts, users can switch between them using the first button found in the Tools overlay. If no specific tool context button is selected, the default `GameObject` tool context remains active.

Within this framework, tools created with the EditorTool API are broadly classified into two main types: Global Tools and Component Tools.

I. Global Tools

Global Tools are custom editor tools that have a broad scope of application within the Unity Editor.

• **Scope**: They affect *any* GameObject in the scene.

• **Availability**: A defining characteristic is that global tools are *always available*, irrespective of the type of GameObject currently selected in the Editor. For instance, Unity's built-in Transform tools (Move, Rotate, Scale, and Rect) are examples of global tools because they can be used with any selected GameObject.

• **Display in UI**: In the Unity Editor's Tools overlay, global tools are displayed in the section immediately following these built-in Transform tools.

II. Component Tools

In contrast, **Component Tools** are custom editor tools with a more specific and contextualised application.

• **Scope**: They are designed to affect a *specific Component* attached to a GameObject.

• **Availability**: Unlike global tools, component tools are *only available* when a GameObject with its associated Component is currently selected in the Scene view or Hierarchy window. For example, a custom manipulator tool specifically designed for lights would only appear and be usable when a GameObject that has a Light component attached to it is selected.

• **Display in UI**: Component tools appear as the last buttons within the Tools overlay. They are further grouped based on the specific component they are designed to interact with. Their visibility and active status are directly tied to the user's active selection.

These distinctions are vital for developers to design intuitive and efficient editor experiences. By understanding whether a tool should operate globally or be tied to specific components, developers can create more contextually relevant and less intrusive custom tools, which contributes to overall productivity and maintainability. Many custom editor tools, such as the

proposed Prefab Painter or Modular Structure Assembly Tool, often integrate with these EditorTool API principles to provide powerful Scene view interactions.
Would you like to delve deeper into how a specific type of custom editor tool, like a Component Tool, is implemented using the EditorTool API and UI Toolkit, or perhaps I could quiz you on the differences between Global and Component Tools?
Discuss what these sources say about Proposed High-Impact Custom Editor Tools (Synty Assets), in the larger context of Unity Editor Tools & UI Toolkit.
Custom Unity Editor Tools are developed to streamline workflows, automate repetitive tasks, and enhance asset management, particularly when dealing with large collections of modular assets like those from Synty Studios. The challenges of managing these assets, which often involve "tedious and time-consuming tasks like sorting, placing, and generating variations of numerous small prefabs," make these custom tools exceptionally valuable. Unity 6.2 also highlights the importance of such tools for enhancing productivity and functionality within the development environment, alongside other features like UI Toolkit.
These high-impact custom editor tools for Synty assets are designed following core architectural principles, primarily leveraging **UI Toolkit** for their user interfaces and **ScriptableObjects** for efficient data management.
Proposed High-Impact Custom Editor Tools for Synty Assets
The sources detail four specific proposed tools to address common pain points when working with Synty Studio's modular asset packs, such as *POLYGON - City Characters*, *POLYGON - Fantasy Characters*, *POLYGON MINI - Fantasy Characters*, *POLYGON - Dungeon Pack*, and *POLYGON - Farm Pack*. Each tool integrates UI Toolkit for its interface and ScriptableObjects for its data architecture, ensuring robustness, maintainability, and a professional user experience.
1. Unified Asset Palette and Search Tool
• **Purpose**: This tool acts as a central hub for discovering, organizing, and quickly accessing all Synty assets within a project, thereby eliminating the need for manual folder navigation. It aims to provide a fast, visual, and searchable interface for the entire Synty collection.
• **Data Architecture (ScriptableObjects)**: It uses a `SyntyPackData` ScriptableObject to store pack names and references to all its prefabs. A `PrefabData` Class, a serializable class nested within `SyntyPackData`, holds details like the prefab reference, its category (e.g., "Characters," "Buildings"), and its pack name. This data is persistent and can be regenerated on demand by a custom Editor script that scans asset folders.
• **UI Toolkit Implementation**: The main user interface is an `EditorWindow`. It features a `ToolbarSearchField` for real-time asset filtering and a `TwoPaneSplitView` to display a list of asset packs on the left and a grid of prefabs on the right. A `ListView` is used for the prefab grid, incorporating custom `makeItem` and `bindItem` functions for UI element creation and data population. Asynchronous thumbnails are generated and displayed for each prefab using `AssetPreview.GetAssetPreview()`.
2. Prefab Painter with Advanced Randomization

• **Purpose**: This tool is designed to rapidly populate scenes with environmental details (e.g., rocks, bushes) from various Synty packs, ensuring natural variation and significantly accelerating prop placement.
• **Data and Implementation**: It relies on a `PrefabBrush` ScriptableObject, which stores a list of prefabs and their associated randomization settings, such as scale range, rotation, and alignment to surface normals. Scene view interaction, including brush preview and mouse input handling, is managed by implementing `OnSceneGUI` within the main Editor script. When painting, the tool instantiates a random prefab from the `PrefabBrush` and applies the configured randomization settings.
• **Synty Specifics**: The tool supports "Mixed Brushes," allowing users to combine props from different Synty packs (e.g., "Nature Pack" rocks with "War Pack" debris). It also includes a "Brush Generator" feature, accessible within the Asset Palette tool, which can create `PrefabBrush` ScriptableObjects by dragging selected assets.
3. Modular Structure Assembly Tool
• **Purpose**: This tool streamlines the assembly of modular structures, such as walls, floors, and roofs, by automatically snapping pieces together based on defined connection points. It is particularly useful for packs like Dungeon, Fantasy Kingdom, and Office.
• **Data and Implementation**: Connection points are established on modular prefabs using empty GameObjects with specific tags or custom `MonoBehaviour` components. An Editor script tracks selected prefabs and facilitates the snapping of new pieces to the nearest available connection point. A `Building Generator` ScriptableObject can further define house layouts, enabling designers to place pre-assembled house variants.
• **UI Toolkit Implementation**: The tool features an `EditorWindow` with a drag-and-drop area for modular pieces and controls for snapping settings and visualizing connection points. A "Build Mode" button activates special scene view interactions, which are handled by `OnSceneGUI`.
4. Character Customization and Randomizer
• **Purpose**: This tool automates the creation of a large number of unique Non-Player Characters (NPCs) by allowing users to mix and match modular character parts from character packs.
• **Data Architecture and Implementation**: It involves a system for categorizing character parts (e.g., body, head, hat, weapons), possibly using `CharacterPart` ScriptableObjects to organize and reference these parts. A "Randomize" button assembles new characters by randomly selecting parts from each category. There's also an option to generate new Prefab Variants of assembled characters for persistent storage.
• **UI Toolkit Implementation**: An `EditorWindow` displays all available character parts and enables users to mix and match them on a preview character. The "Randomize" button triggers the character assembly process.
Integration with Unity Editor Tools & UI Toolkit
These proposed tools exemplify the modern approach to Unity Editor extension development.

• **UI Toolkit**: All these tools are designed with UI Toolkit for their interfaces, leveraging its visual authoring capabilities via `UI Builder`. This ensures "intuitive and professional editor interfaces" and benefits from UI Toolkit's event-driven performance and flexible styling with `USS`.

• **ScriptableObjects**: Data management for all proposed tools heavily relies on ScriptableObjects for persistence, decoupling data from logic, and shareability across different parts of the tool or project. This ensures tool data like asset lists, randomization settings, connection points, and character part categories are robustly managed.

• **EditorWindow and OnSceneGUI**: The tools utilize `EditorWindow` for their main interfaces, ensuring they are available at all times, independent of selected GameObjects. Interactions directly within the Scene view are handled by `OnSceneGUI`, particularly for tools like the Prefab Painter and Modular Structure Assembly Tool.

Commercial Potential

These proposed tools are also considered for commercial release on the Unity Asset Store, which necessitates a focus on user experience, marketing, and comprehensive documentation. This reinforces the need for "robust, maintainable, and marketable" tools built on solid architectural principles.

Would you like to explore the specific UI Toolkit components used in one of these proposed tools, such as the `TwoPaneSplitView` or `ListView`, or perhaps discuss the challenges and benefits of releasing such tools on the Unity Asset Store?

Unity Editor Tools for Synty Assets

(Saved responses are view only)

Custom Unity Editor Tools are developed to streamline workflows, automate repetitive tasks, and enhance asset management, particularly when dealing with large collections of modular assets like those from Synty Studios. The challenges of managing these assets, which often involve "tedious and time-consuming tasks like sorting, placing, and generating variations of numerous small prefabs," make these custom tools exceptionally valuable. Unity 6.2 also highlights the importance of such tools for enhancing productivity and functionality within the development environment, alongside other features like UI Toolkit.

These high-impact custom editor tools for Synty assets are designed following core architectural principles, primarily leveraging **UI Toolkit** for their user interfaces and **ScriptableObjects** for efficient data management.

Proposed High-Impact Custom Editor Tools for Synty Assets

The sources detail four specific proposed tools to address common pain points when working with Synty Studio's modular asset packs, such as *POLYGON - City Characters*, *POLYGON - Fantasy Characters*, *POLYGON MINI - Fantasy Characters*, *POLYGON - Dungeon Pack*, and *POLYGON - Farm Pack*. Each tool integrates UI Toolkit for its interface and ScriptableObjects for its data architecture, ensuring robustness, maintainability, and a professional user experience.

1. Unified Asset Palette and Search Tool

• **Purpose**: This tool acts as a central hub for discovering, organizing, and quickly accessing all Synty assets within a project, thereby eliminating the need for manual folder navigation. It aims to provide a fast, visual, and searchable interface for the entire Synty collection.

• **Data Architecture (ScriptableObjects)**: It uses a `SyntyPackData` ScriptableObject to store pack names and references to all its prefabs. A `PrefabData` Class, a serializable class nested within `SyntyPackData`, holds details like the prefab reference, its category (e.g., "Characters," "Buildings"), and its pack name. This data is persistent and can be regenerated on demand by a custom Editor script that scans asset folders.
• **UI Toolkit Implementation**: The main user interface is an `EditorWindow`. It features a `ToolbarSearchField` for real-time asset filtering and a `TwoPaneSplitView` to display a list of asset packs on the left and a grid of prefabs on the right. A `ListView` is used for the prefab grid, incorporating custom `makeItem` and `bindItem` functions for UI element creation and data population. Asynchronous thumbnails are generated and displayed for each prefab using `AssetPreview.GetAssetPreview()`.
2. Prefab Painter with Advanced Randomization
• **Purpose**: This tool is designed to rapidly populate scenes with environmental details (e.g., rocks, bushes) from various Synty packs, ensuring natural variation and significantly accelerating prop placement.
• **Data and Implementation**: It relies on a `PrefabBrush` ScriptableObject, which stores a list of prefabs and their associated randomization settings, such as scale range, rotation, and alignment to surface normals. Scene view interaction, including brush preview and mouse input handling, is managed by implementing `OnSceneGUI` within the main Editor script. When painting, the tool instantiates a random prefab from the `PrefabBrush` and applies the configured randomization settings.
• **Synty Specifics**: The tool supports "Mixed Brushes," allowing users to combine props from different Synty packs (e.g., "Nature Pack" rocks with "War Pack" debris). It also includes a "Brush Generator" feature, accessible within the Asset Palette tool, which can create `PrefabBrush` ScriptableObjects by dragging selected assets.
3. Modular Structure Assembly Tool
• **Purpose**: This tool streamlines the assembly of modular structures, such as walls, floors, and roofs, by automatically snapping pieces together based on defined connection points. It is particularly useful for packs like Dungeon, Fantasy Kingdom, and Office.
• **Data and Implementation**: Connection points are established on modular prefabs using empty GameObjects with specific tags or custom `MonoBehaviour` components. An Editor script tracks selected prefabs and facilitates the snapping of new pieces to the nearest available connection point. A `Building Generator` ScriptableObject can further define house layouts, enabling designers to place pre-assembled house variants.
• **UI Toolkit Implementation**: The tool features an `EditorWindow` with a drag-and-drop area for modular pieces and controls for snapping settings and visualizing connection points. A "Build Mode" button activates special scene view interactions, which are handled by `OnSceneGUI`.
4. Character Customization and Randomizer
• **Purpose**: This tool automates the creation of a large number of unique Non-Player Characters (NPCs) by allowing users to mix and match modular character parts from character packs.
• **Data Architecture and Implementation**: It involves a system for categorizing character parts (e.g., body, head, hat, weapons), possibly using `CharacterPart` ScriptableObjects to organize and reference these parts. A "Randomize" button assembles new characters by randomly selecting parts from each category. There's also an option to generate new Prefab Variants of assembled characters for persistent storage.

• **UI Toolkit Implementation**: An `EditorWindow` displays all available character parts and enables users to mix and match them on a preview character. The "Randomize" button triggers the character assembly process.
Integration with Unity Editor Tools & UI Toolkit

These proposed tools exemplify the modern approach to Unity Editor extension development.
• **UI Toolkit**: All these tools are designed with UI Toolkit for their interfaces, leveraging its visual authoring capabilities via `UI Builder`. This ensures "intuitive and professional editor interfaces" and benefits from UI Toolkit's event-driven performance and flexible styling with `USS`.
• **ScriptableObjects**: Data management for all proposed tools heavily relies on ScriptableObjects for persistence, decoupling data from logic, and shareability across different parts of the tool or project. This ensures tool data like asset lists, randomization settings, connection points, and character part categories are robustly managed.
• **EditorWindow and OnSceneGUI**: The tools utilize `EditorWindow` for their main interfaces, ensuring they are available at all times, independent of selected GameObjects. Interactions directly within the Scene view are handled by `OnSceneGUI`, particularly for tools like the Prefab Painter and Modular Structure Assembly Tool.
Commercial Potential

These proposed tools are also considered for commercial release on the Unity Asset Store, which necessitates a focus on user experience, marketing, and comprehensive documentation. This reinforces the need for "robust, maintainable, and marketable" tools built on solid architectural principles.
Would you like to explore the specific UI Toolkit components used in one of these proposed tools, such as the `TwoPaneSplitView` or `ListView`, or perhaps discuss the challenges and benefits of releasing such tools on the Unity Asset Store?

Unity Editor Tools and UI Toolkit Briefing

1. Introduction to Custom Unity Editor Tools

Custom Unity Editor tools are developed to enhance productivity, automate repetitive tasks, and improve overall efficiency within the Unity development environment. These tools are particularly beneficial when managing extensive collections of modular assets, such as those from Synty Studios, which often entail "tedious and time-consuming tasks like sorting, placing, and generating variations of numerous small prefabs." The Unity 6.2 release further emphasises the significance of such tools by highlighting improvements in "profiling options, ProBuilder, Cinemachine, and UI Toolkit" to boost productivity.

2. Core Architectural Principles for Editor Tools

Building robust, maintainable, and marketable custom Unity Editor tools requires adherence to several core architectural principles.

2.1 User Interface (UI): UI Toolkit

UI Toolkit is Unity's modern UI framework, designed to replace both the older Immediate Mode GUI (IMGUI) for editor interfaces and eventually Unity UI (uGUI) for runtime interfaces. It is "heavily influenced by web technologies like CSS and HTML" but specifically tailored for Unity's requirements.
Key Advantages over IMGUI:

FeatureUI ToolkitIMGUI (Immediate Mode GUI)AuthoringSupports visual authoring using the UI Builder, separating visuals (UXML/USS) from logic (C#). This is analogous to HTML/CSS separation from JavaScript. In Unity 6, data binding in UI Builder makes linking ScriptableObjects or data easier.Requires UI to be entirely coded in C#, leading to "messy and unreadable UIs for complex tools."PerformanceEvent-driven, meaning it "only redraws elements when changes occur," making it highly "efficient." This also extends to controls like ListView and Multicolumn ListView, which are "virtualized containers" that "only creates elements for what can be visible at a time," and then rebinds data to those elements as the user scrolls, leading to superior performance for large datasets."Redraws the entire UI every frame," which is "resource-intensive" and can lead to poorer performance for complex interfaces.StylingUtilises USS (Unity Style Sheets) for "flexible and powerful styling," promoting consistent design. USS syntax is "the same as CSS syntax, but USS includes overrides and customizations to work better with Unity." It supports editor variables for theme consistency (e.g., light/dark mode) and allows for transition animations and transforms.Relies on line-by-line code styling, hindering consistent design.Best Use CasesIdeal for "polished, complex, professional editor tools for large projects," as well as for runtime UI (with Unity 2023 introducing a new binding system).Better suited for "simple debug windows or quick inspectors."UI Toolkit Components & Features:

UI Builder: A visual authoring tool within the Unity Editor for creating and editing UI Documents (.uxml) and StyleSheets (.uss). It enables drag-and-drop UI creation and real-time preview, supporting "editor extension authoring" for additional controls and access to the editor theme.
UXML (Unity Extensible Markup Language): An XML-based asset that defines the structure and layout of UI elements, similar to HTML. UXML assets can be "instantiate[d] existing UXML Documents as Templates inside your UXML Documents as Template Instances, similar to how Prefabs work in Unity."
USS (Unity Style Sheets): CSS-like text files for flexible and powerful styling. While variables cannot be created directly in UI Builder, they can be assigned to style properties once defined in a text editor.
VisualElement: The base class for all UI Toolkit elements, serving as a container and parent.
Data Binding: Links UI controls to object properties or serialized properties, enabling automatic display and updates of values. This two-way connection is "useful when you create custom Inspector windows." Unity 6.1 introduced enhanced data binding.
Events: An event system for handling user interactions and notifications, with terminology and naming similar to HTML events. Callbacks can be registered for various events, such as button clicks or value changes.
Custom Property Drawers & Editors: UI Toolkit allows direct implementation of custom property drawers and full custom editors, providing granular control over data presentation in the Inspector. This is a significant improvement over previous methods that required fallbacks for IMGUI.
Live Reload: Provides "immediate UI updates in the editor during development," even without saving UXML/USS files, significantly speeding up workflow.
Editor Foundations Design System: A Unity resource offering "helpful guidance, best practices, and other resources to help users build a better Editor UI," promoting consistency with Unity's native interface.
Reusable Components: By extending VisualElement (or BindableElement) and defining UI in UXML, developers can create reusable UI components that "encapsulate behaviors and interactions," saving development time. These components require a UXML Factory to be exposed as a tag and UXML Traits for attributes.
ListView & Multicolumn ListView: Controls designed for efficient display of large lists of data by pooling and recycling elements as the user scrolls.

2.2 Data Management: ScriptableObjects

ScriptableObjects are "serializable Unity type derived from UnityEngine.Object" and are fundamental for storing tool data, such as asset lists or configuration settings. They exist "in the project as assets, independent of GameObjects."

Benefits:

Persistence: "Data is saved as a .asset file, retaining information across Unity sessions."
Decoupling: "Separates tool data from its logic, allowing UI windows to be closed without data loss."
Shareability: "Enables multiple tools or systems to reference and share the same data," reducing memory usage by avoiding copies.

Use Cases:

Commonly used as "containers for shared data used by multiple objects at runtime" and for "saving and storing data during an Editor session." Examples include PrefabPaletteData, SyntyPackData, and PrefabBrush for custom tools.
Saving Changes:

When modifying ScriptableObjects via script in Edit mode, EditorUtility.SetDirty() "must be called to ensure Unity's serialization system recognizes and saves the changes to disk."

2.3 Serialized Objects and Properties

SerializedObject and SerializedProperty are fundamental for generically editing serialized fields on Unity objects. They are the "recommended approach as opposed to modifying inspected target objects directly" for creating custom Editors.
Benefits:

- Automatic Handling: They "automatically handle dirtying individual serialized fields so they will be processed by the Undo system and styled correctly for Prefab overrides when drawn in the Inspector."
- Multi-Object Editing: SerializedObject "opens a data stream to one or more target Unity objects at a time, which allows you to simultaneously edit serialized data that the objects share in common."
- Workflow: Changes made via SerializedProperty within a SerializedObject stream must be flushed using SerializedObject.ApplyModifiedProperties() to persist. SerializedObject.Update() should be called before reading data if a reference is maintained over multiple frames. Note that ApplyModifiedProperties "will not respect any data validation logic you may have in property setters."

3. Types of Custom Editor Tools and Their Context

Unity provides the EditorTool API for creating custom tools that operate within the Editor, categorised by their context and scope.

3.1 Tool Context

The EditorToolContext API "changes what the Editor's built-in Transform tools affect." The default context is GameObject, affecting a GameObject's Transform values. Other contexts, like the Spline tool, can affect different elements. If multiple tool contexts exist, users can select them via the Tools overlay.

3.2 Global Tools vs. Component Tools

- Global Tools: "Affect any GameObject" and are "always available regardless of the type of GameObject you select" (e.g., built-in Transform tools like Move, Rotate, Scale, Rect).
- Component Tools: "Affect a specific component" and are "only available when you select a GameObject with the component the tool comes from attached to it" (e.g., a custom manipulator for lights is available only when a Light component is selected).

4. Proposed High-Impact Custom Editor Tools for Modular Assets

Several high-impact tools are proposed to address workflow challenges, especially when working with modular asset packs like Synty Studios.

4.1 Unified Asset Palette and Search Tool

Purpose: "A central hub for discovering, organizing, and quickly accessing all Synty assets within a project," eliminating manual folder navigation.
Data Architecture: Uses SyntyPackData ScriptableObject (stores pack names and prefab references) and PrefabData Class (serializable class within SyntyPackData for prefab reference, category, and pack name). Data is persistent and regeneratable.
UI Toolkit Implementation: EditorWindow as the main UI, ToolbarSearchField for filtering, TwoPaneSplitView to display packs and a prefab grid, and ListView for the prefab grid (with custom makeItem and bindItem functions). AssetPreview.GetAssetPreview() generates asynchronous thumbnails.

4.2 Prefab Painter with Advanced Randomization

Purpose: To "rapidly populate scenes with environmental details (e.g., rocks, bushes) from various Synty packs, while ensuring natural variation."
Data and Implementation: Uses a PrefabBrush ScriptableObject (stores prefabs and randomization settings like scale, rotation, alignment to surface normal). OnSceneGUI handles brush preview and mouse input, instantiating random prefabs with configured settings.
Synty Specifics: Supports "Mixed Brushes" (combining props from different packs) and a "Brush Generator" feature within the Asset Palette tool (creates PrefabBrush ScriptableObjects by dragging selected assets).

4.3 Modular Structure Assembly Tool

Purpose: To "streamline the assembly of modular walls, floors, and roofs by automatically snapping pieces together based on defined connection points."
Data and Implementation: Connection points are defined on modular prefabs (empty GameObjects with specific tags or custom MonoBehaviour components). An editor script tracks selected prefabs and snaps new pieces. A Building Generator ScriptableObject can define house layouts for pre-assembled variants.
UI Toolkit Implementation: EditorWindow with a drag-and-drop area for modular pieces, controls for snapping settings, visualization of connection points, and a "Build Mode" button activating OnSceneGUI interactions.

4.4 Character Customization and Randomizer

Purpose: To "quickly generate a large number of unique non-player characters (NPCs) by mixing and matching modular character parts."

Data Architecture and Implementation: A system for categorizing character parts (body, head, hat, weapons), potentially using CharacterPart ScriptableObjects. A "Randomize" button assembles new characters by selecting parts from each category, with an option to generate new Prefab Variants for persistent storage.
UI Toolkit Implementation: EditorWindow displays character parts, allows mix-and-match on a preview character, and features a "Randomize" button for assembly.

5. Preparing for Commercial Release on the Unity Asset Store

For tools intended for commercial release, a focus on user experience, marketing, and ongoing support is crucial. The Unity Asset Store allows developers to "earn while doing what you love," "reach over 1.7 million monthly Asset Store users," and "own your content."

5.1 Distribution Process

Register Publisher Profile: Register on the Publisher Portal with a Unity ID, providing portfolio and business information.
Create and Upload Asset: Create a package on the Publisher Portal following Submission Guidelines and upload files from the Unity Editor using Asset Store Tools.
Submit for Review: Unity’s curation team reviews assets, accepting them or providing feedback for resubmission.

5.2 Professional Store Page

The Asset Store page is the primary marketing interface and requires:

Key Art: "Clean, professional, and clear communication of the tool's function" (icon, card, cover).
Screenshots and GIFs: "High-quality visuals demonstrating the tool in action."
Video Trailer: "A short (1-2 minute) video showcasing the problem, the tool's solution, and key features."

5.3 Comprehensive Documentation

Documentation is mandatory for assets with code or setup requirements and should be "included as PDF, Markdown (.md), or text file within the package." Content must include:

Installation & Setup guide.
Quick Start Guide for main features.
Detailed breakdown of all features, buttons, and settings.
Contact information for support.

5.4 Legal Considerations

Publishers must agree to the "Asset Store Provider Agreement" and the "Asset Store Terms of Service and EULA." It is essential to ensure that the publisher has "all rights to the content and can distribute it on the Asset Store," removing any questionable content before submission.

6. Additional Tools and Concepts

NaughtyAttributes: An open-source Unity Inspector extension that "expands the range of attributes that Unity provides so that you can create powerful inspectors without the need of custom editors or property drawers."
EditorWindow: A class for creating custom editor windows that are available "at all times, no matter what my selection is," unlike Inspector windows.
Runtime UI with UI Toolkit: UI Toolkit is designed for both editor and runtime UI. Unity 2023 introduced a new binding system for runtime UI, allowing DataSource and BindingPath attributes to work without extensive manual C# code.
Game Design Document (GDD): A comprehensive document outlining all aspects of a game, including game description, functional specifications, user interface, art, sound, story, level requirements, technical specifications, and production schedule. Custom editor tools are designed to enhance the development process outlined in such documents.
Element Management Best Practices: To improve performance and stability when managing elements, it is recommended to "pre-create hierarchies," "pool recurring elements," and "keep the number of visible elements low" by using ListView when possible.